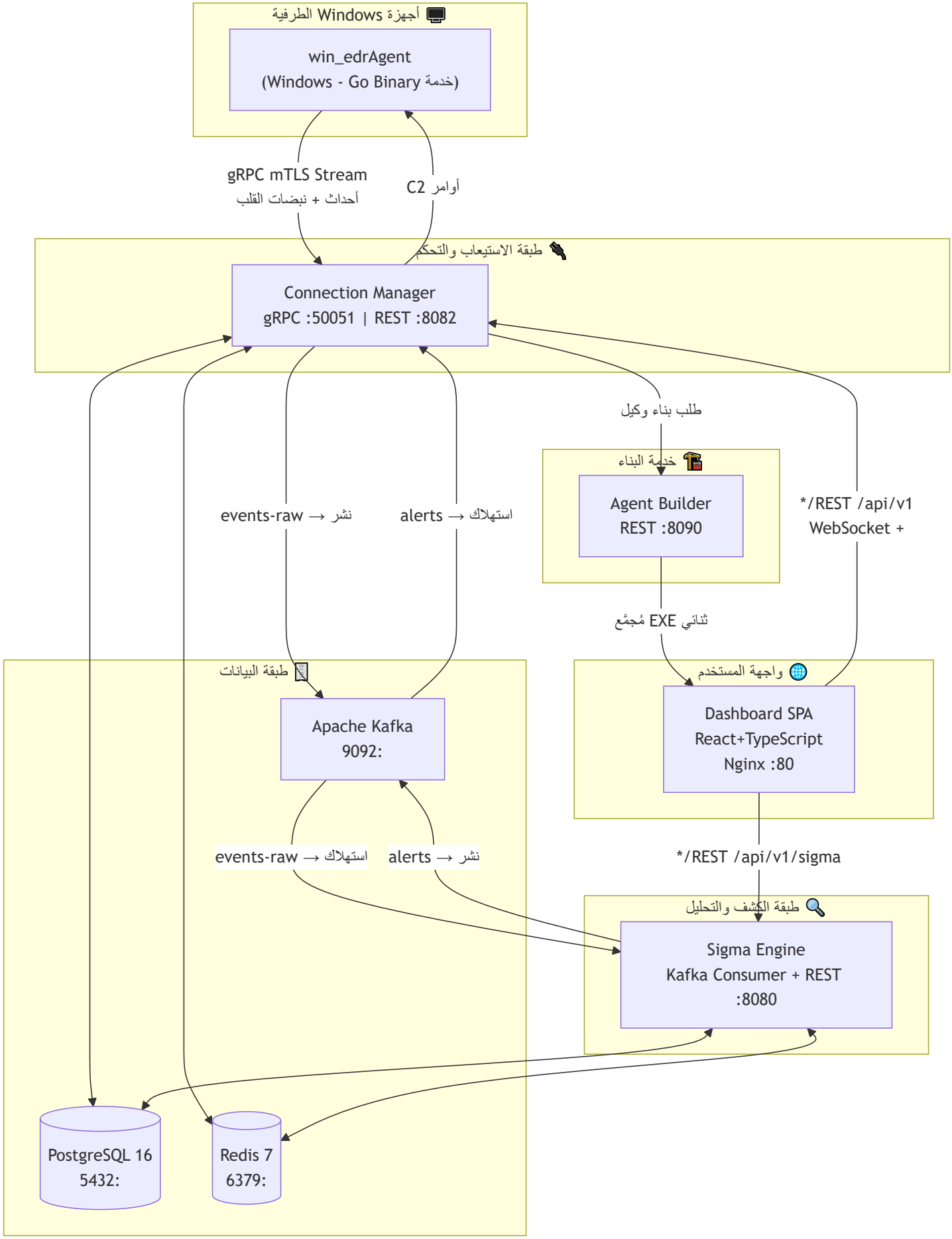
الملف الثاني: المعمارية الشاملة للنظام — كيف تتحدث المكوّنات مع بعضها

أولاً: نظرة هيكلية على المعمارية


أجهزة Windows الطرفية
win_edrAgent
(Windows - Go Binary خدمة)
gRPC mTLS Stream
أحداث + نبضات القلب
أوامر C2
طبقة الاستيعاب والتحكم
Connection Manager
gRPC :50051 | REST :8082
طلب بناء وكيل
خدمة البناء
Agent Builder
REST :8090
*/REST /api/v1
WebSocket +
events-raw → نشر
alerts → استهلاك
ثنائي EXE مُجمَّع
طبقة البيانات
Apache Kafka
9092:
واجهة المستخدم
Dashboard SPA
React+TypeScript
Nginx :80
events-raw → استهلاك
alerts → نشر
*/REST /api/v1/sigma
طبقة الكشف والتحليل
Sigma Engine
Kafka Consumer + REST
:8080
PostgreSQL 16
5432:
Redis 7
6379:

ثانياً: طبقات المعمارية (Layered Architecture)

النظام مبني على معمارية متعددة الطبقات، حيث تختص كل طبقة بمسؤولية محددة وتتواصل مع الطبقات المجاورة فقط.

الطبقة الأولى — طرف الجهاز (Endpoint Layer): هي الوكيل المنشور على جهاز Windows. مسؤولة عن جمع الأحداث الأمنية من نواة نظام التشغيل وإرسالها للخادم. تعمل بشكل مستقل حتى عند انقطاع الشبكة عن طريق التخزين المحلي.

الطبقة الثانية — طبقة الاستيعاب (Ingestion Layer): هي Connection Manager. مسؤولة عن استقبال الاتصالات من الوكلاء، التحقق من هوياتهم، استيعاب أحداثهم، وتوزيعها على الطبقة التالية. أيضاً مسؤولة عن إدارة دورة حياة الوكلاء والأوامر.

الطبقة الثالثة — طبقة النقل (Message Bus Layer): هي Kafka. تعمل كناقل غير متزامن بين الاستيعاب والكشف. تضمن عدم فقدان الأحداث وتسمح بمعالجة موازية.

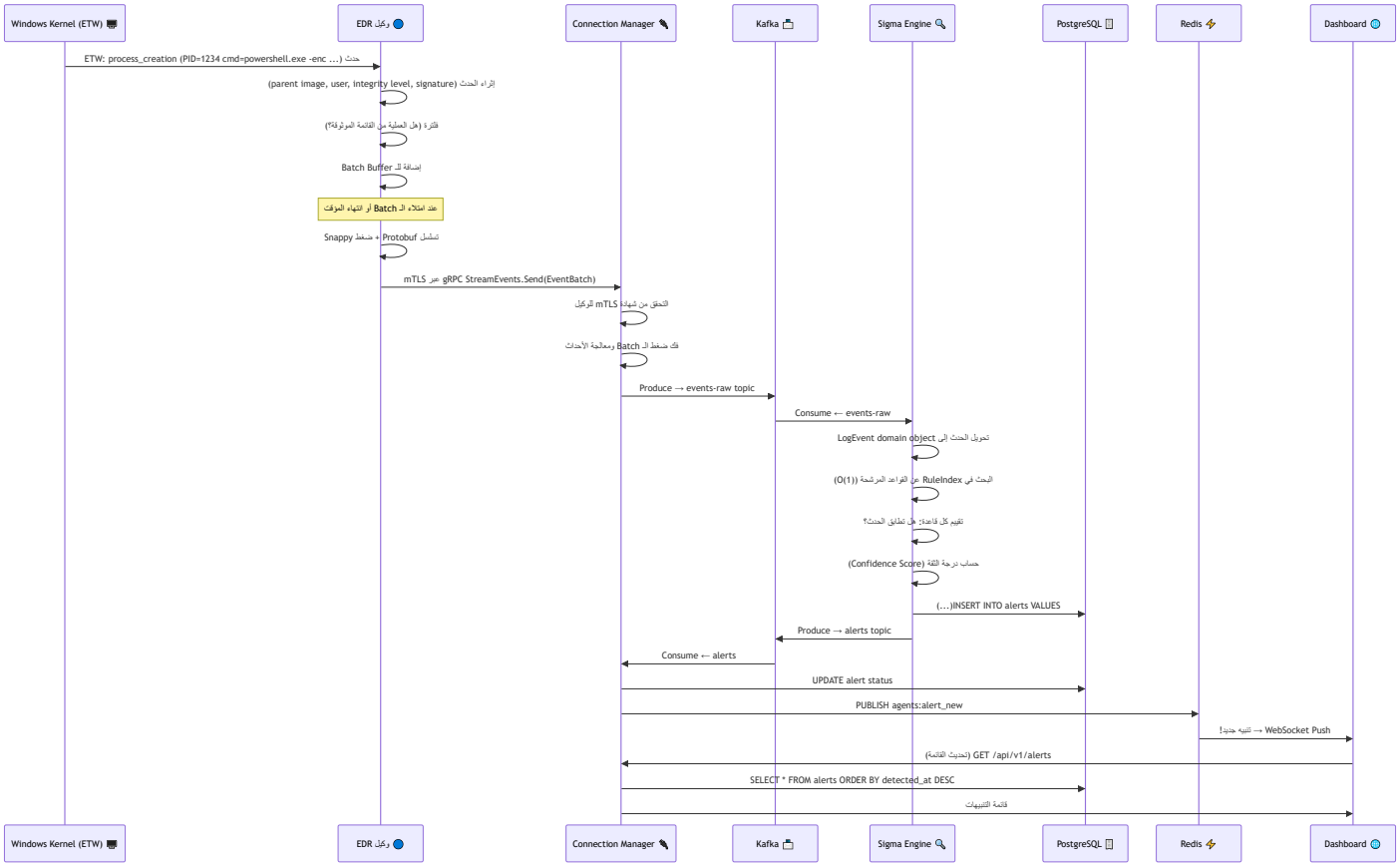
الطبقة الرابعة — طبقة الكشف (Detection Layer): هي Sigma Engine. تستهلك الأحداث من Kafka، تُطابقها مع قواعد Sigma، وتُنتج تنبيهات عند اكتشاف تهديد.

الطبقة الخامسة — طبقة التخزين (Data Layer): PostgreSQL وRedis. تحتفظ بكل البيانات المستمرة والمؤقتة.

الطبقة السادسة — طبقة العرض (Presentation Layer): لوحة التحكم React. تُترجم البيانات إلى واجهة مرئية يتفاعل معها المحلل الأمني.

ثالثاً: تدفق البيانات الكامل من البداية للنهاية

لنتتبع رحلة حدث أمني واحد من لحظة وقوعه على الجهاز حتى ظهوره كتنبيه في لوحة التحكم:



هذه الرحلة كاملة تستغرق في المتوسط أقل من ثانيتين من لحظة وقوع الحدث على الجهاز حتى ظهور التنبيه في لوحة التحكم.

رابعاً: معمارية الاتصال gRPC بالتفصيل

الاتصال بين الوكيل ومدير الاتصالات يتم عبر gRPC ببروتوكول mTLS. لنفهم كيف يعمل هذا بالضبط:

mTLS (Mutual TLS) يعني أن كلاً من الوكيل والخادم يُقدّمان شهادة رقمية للإثبات هويتهما. عند بناء الاتصال:

- الخادم يُقدّم شهادته ليثبت للوكيل أنه الخادم الشرعي.
- الوكيل يُقدّم شهادته (المُصدَرة له عند التسجيل) ليثبت للخادم أنه وكيل مُعتمَد.
- الشهادتان موقَّعتان من CA (Certificate Authority) خاص بالمنصة.
- أي شهادة لم تُوقِّعها هذه CA ترفضها الطبقة الآمنة تلقائياً.

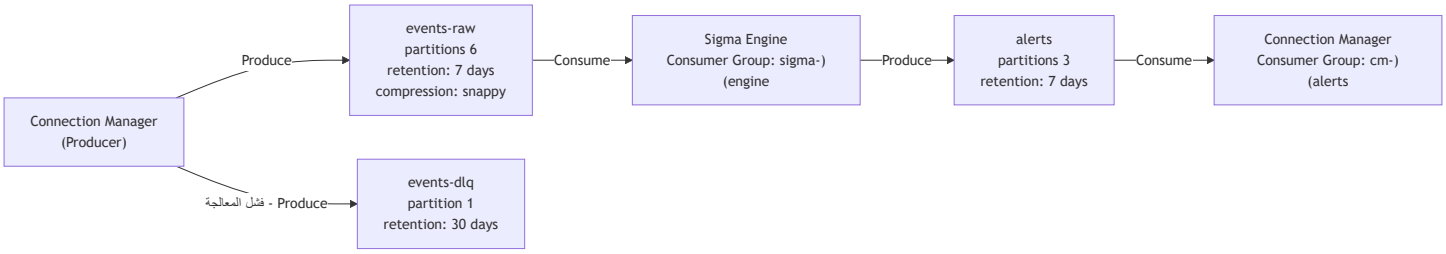
يستخدم النظام قناتين للاتصال:

القناة الأولى — Streaming (طويلة الأمد): قناة مفتوحة باستمرار بين الوكيل والخادم. الوكيل يُرسل عليها دُفعات الأحداث، والخادم يُرسل عليها أوامر C2. هذه القناة تعمل على مبدأ Bidirectional Streaming، مما يعني أن كلا الطرفين يمكنه الإرسال في أي وقت دون انتظار دوره.

القناة الثانية — Unary RPC (طلب-استجابة): تُستخدَم لنبضات القلب (Heartbeats). كل 30 ثانية يُرسل الوكيل حزمة Heartbeat تتضمن: معدل CPU، الذاكرة المستخدمة، عدد الأحداث المُنتَجة والمُرسَلة والمُسقَطة، حالة Sysmon، وعناوين IP. الخادم يرد بـ: هل هناك

أوامر معلّقة؟ هل الشهادة بحاجة للتجديد؟ هل هناك تحديث للإعدادات؟

خامساً: معمارية Kafka وتصميم الـ Topics



تصميم الـ topics له منطق دقيق. `events-raw` لديه 6 partitions لأن الأحداث تأتي بكميات كبيرة وتحتاج معالجة موازية. كل partition يُخصَّص لمجموعة من الوكلاء (يُستخدَم `agent_id` كـ partition key) مما يضمن أن أحداث نفس الوكيل تذهب دائماً لنفس الـ partition وبالترتيب الزمني الصحيح. `alerts` لديه 3 partitions لأن التنبيهات أقل كمية بكثير.

سادساً: معمارية قاعدة البيانات

قاعدة البيانات PostgreSQL تحتوي على مجموعتين من الجداول: جداول connection-manager وجداول sigma-engine. وعند بدء تشغيل Docker Compose، يُنفَّذ كل من مجلد migrations بالترتيب لبناء المخطط الكامل.

الجداول الرئيسية في connection-manager:

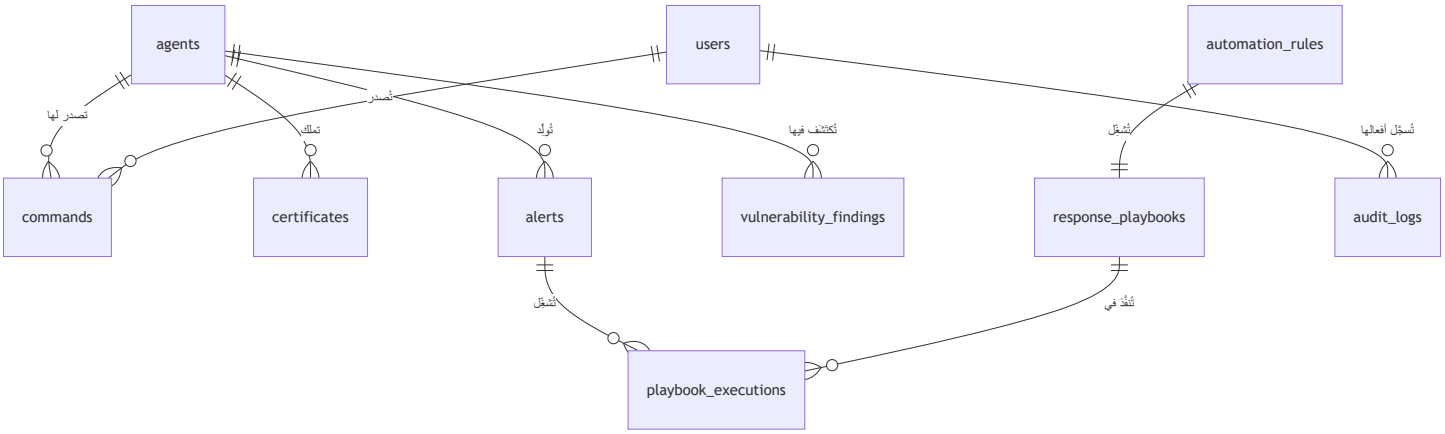
- `agents` : تخزّن بيانات كل وكيل مُسجَّل (hostname, status, OS, hardware_id, الشهادة، المقاييس، درجة الصحة).
- `users` : بيانات محللي SOC (username, password_hash bcrypt, role, حالة القفل).
- `commands` : الأوامر الصادرة للوكلاء وحالتها (pending/sent/completed/failed).
- `certificates` : الشهادات الرقمية الصادرة لكل وكيل.
- `audit_logs` : سجل كامل لكل فعل إداري في النظام.
- `enrollment_tokens` : رموز التسجيل مع حد أقصى للاستخدام وتاريخ انتهاء.
- `quarantine_items` : الملفات المحجورة مع المسار الأصلي ومسار الحجر ومعلومات التهديد.
- `response_playbooks` : تعريفات Playbooks للاستجابة.
- `automation_rules` : قواعد الأتمتة التي تُشغّل Playbooks آلياً.
- `vulnerability_findings` : الثغرات الأمنية المكتشفة في كل جهاز.

الجداول الرئيسية في sigma-engine:

- `sigma_rules` : قواعد الكشف المخزّنة في قاعدة البيانات (يمكن تعديلها من لوحة التحكم).
- `alerts` : التنبيهات الأمنية المُنتَجة (مع risk_score وdegree of confidence ومعلومات MITRE).

- `raw_events` : عيّنات من الأحداث الخام (لأغراض التحقيق الجنائي).

العلاقات بين الجداول:



سابعاً: معمارية الأمان (Security Architecture)

النظام يُطبّق مبدأ Defense in Depth — الدفاع بالعمق — عبر طبقات أمان متعددة.

طبقة الشبكة: جميع الاتصالات بين الوكيل والخادم مُشفَّرة بـ TLS 1.2 كحد أدنى. mTLS يتحقق من هوية الطرفين. لا يوجد اتصال HTTP عادي بين الوكيل والخادم.

طبقة الهوية: كل وكيل يحصل على شهادة X.509 فريدة موقّعة من CA خاص بالمنصة. الشهادة تُخزَّن مشفّرةً في Windows Registry (ليس في ملفات يمكن قراءتها بسهولة). ACLs على مفتاح Registry تمنع أي مستخدم غير SYSTEM أو ADMIN من قراءته.

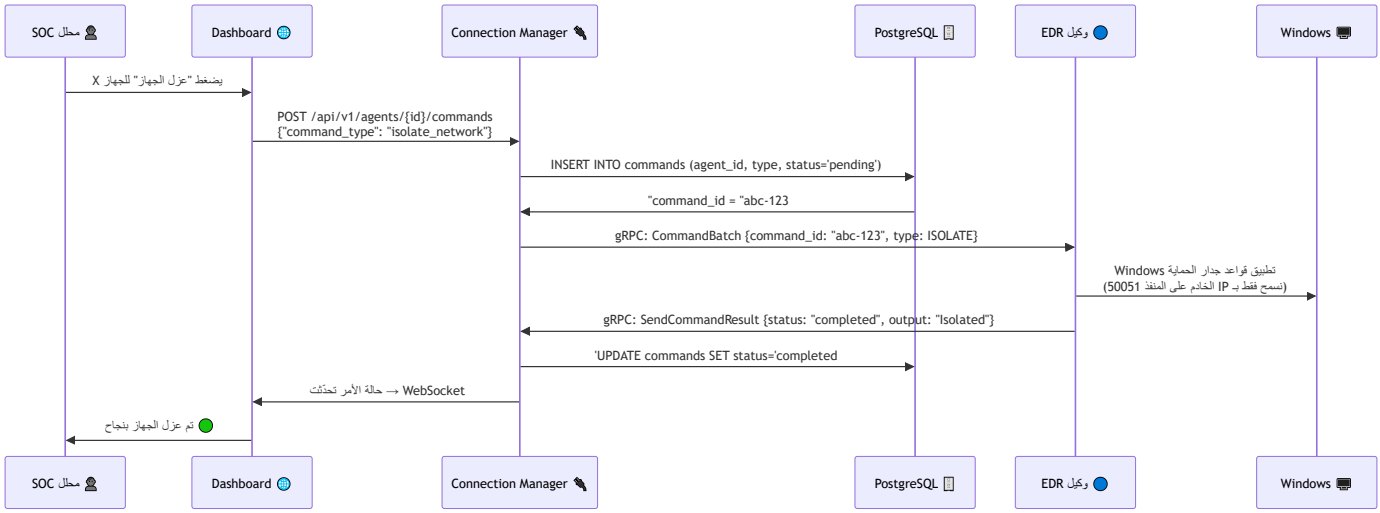
طبقة المصادقة لواجهة API: المحللون يُسجّلون دخولهم بكلمة مرور مُشفَّرة بـ bcrypt. بعد النجاح يحصلون على JWT token يُستخدَم في كل الطلبات التالية. الـ JWT مُوقَّع بمفتاح RSA خاص. عند تسجيل الخروج يُضاف الرمز للقائمة السوداء في Redis.

آلية حماية ضد قفل الحساب: بعد 5 محاولات تسجيل دخول فاشلة، يُقفَل الحساب لمدة 15 دقيقة تلقائياً. هذا يحمي من هجمات Brute Force.

طبقة حماية الوكيل (Anti-Tamper): الوكيل يحمي نفسه من التلاعب. يُنشئ ACLs على مجلد `C:\ProgramData\EDR` تمنع المستخدمين العاديين من تعديل ملفاته أو حذفها.

ثامناً: معمارية الـ C2 (Command and Control)

عندما يُصدِر محلل أمري لجهاز طرفي من لوحة التحكم، إليك ما يحدث بالضبط:



الأوامر المدعومة تشمل: إيقاف عملية (kill_process)، حجر ملف (quarantine_file)، عزل الشبكة (isolate_network)، استعادة الشبكة (restore_network)، تشغيل أمر CMD (run_cmd)، جمع السجلات (collect_logs)، فحص الذاكرة (scan_memory)، حظر IP (block_ip)، حظر نطاق (block_domain)، تحديث التوقيعات (update_signatures)، إزالة الوكيل (uninstall_agent).

تاسعاً: معمارية الـ Nginx كـ Reverse Proxy

لوحة التحكم تعمل داخل Docker container يحتوي على Nginx. الـ Nginx يُوزّع الطلبات كما يلي:

- الطلبات إلى `*/api/v1/sigma/` تُعاد كتابتها وتُوجَّه إلى `sigma-engine:8080` (الخدمة الداخلية في Docker).
- الطلبات إلى `*/api/v1/` تُوجَّه إلى `host.docker.internal:30082` (مدير الاتصالات على المضيف).
- طلبات WebSocket إلى `*/ws/` تُوجَّه مع ترقية البروتوكول لـ `Upgrade: websocket` .
- كل الطلبات الأخرى تُوجَّه إلى ملفات React الثابتة في `usr/share/nginx/html/` .

هذا التصميم يعني أن المتصفح يتعامل مع عنوان واحد فقط (المنفذ 30088) ويبدو أن كل الـ APIs تأتي من نفس المصدر، مما يتجنب مشاكل CORS بشكل كامل.